

Event-Driven Network Simulator

Documentazione tecnica — Phase 1

Verso reti agentiche: gestione autonoma della congestione

Tesi Magistrale — Flavio Bianco

Giugno 2026

Contenuti del documento

#	Sezione	Argomento
1	Contesto e obiettivi	Le 4 fasi della tesi, perche serve un simulatore
2	Architettura del sistema	Moduli Python, design pattern, dipendenze
3	Il motore a eventi discreti	14 tipi di evento, priority queue, ciclo di vita del pacchetto
4	Rete e code	Topology, Node, Link, QueueManager, modello M/D/1
5	Traffic e flussi	FlowModel, TrafficClass, Packet, priorit�
6	State machine di congestione	5 stati, soglie configurabili, azioni correttive
7	Metriche di performance	Throughput, PDR, latenza, fairness, drop count
8	Scenari canonici	6 scenari di test, parametri, topologia
9	Risultati e validazione	Demo Phase 1, analisi M/D/1, stati osservati
10	Prossimi passi	Phase 2 rule-based, MARL, Agentic AI

1. Contesto e obiettivi della tesi

Il simulatore e' il cuore tecnico di una tesi magistrale il cui titolo e' "**Towards Agentic Networks: Autonomous Congestion Management**". L'obiettivo finale e' costruire un sistema di rete capace di gestire autonomamente la congestione senza intervento umano, sfruttando progressivamente tecniche sempre piu' sofisticate: da regole euristiche semplici fino a modelli basati su Large Language Models con capacita' agentiche.

La tesi e' strutturata in **4 fasi progressive**, ognuna costruita sopra la precedente. Il simulatore sviluppato in Phase 1 e' la piattaforma sperimentale comune a tutte le fasi: permette di testare e confrontare le diverse strategie in un ambiente controllato e riproducibile.

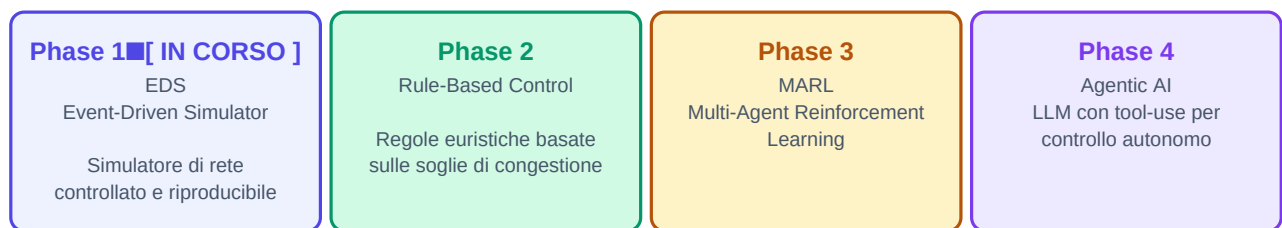


Fig. 1 — Roadmap delle 4 fasi della tesi. Phase 1 (simulatore) e' il fondamento di tutte le fasi successive.

Perche' un simulatore event-driven?

Un simulatore a **eventi discreti** (Discrete Event Simulation, DES) e' il modello standard per lo studio di sistemi di rete. I motivi principali:

- **Riproducibilita'**: con lo stesso seed casuale si ottengono esattamente gli stessi risultati — fondamentale per comparare algoritmi diversi in condizioni identiche.
- **Controllo completo**: si possono iniettare eventi arbitrari (guasti, surge di traffico, degradazione di banda) in momenti precisi del tempo simulato.
- **Nessuna dipendenza da reti fisiche**: si possono testare scenari estremi (rete sotto stress massimo, multiple failure simultanee) senza rischi.
- **Velocita'**: 100 secondi di simulazione di rete vengono completati in frazioni di secondo reale.

Cosa NON e' questo simulatore

E' importante chiarire cosa il simulatore *non* fa. **Non** usa librerie di simulazione esterne (come ns-3, OMNeT++ o SimPy). Tutto il motore a eventi, il modello di rete, le code e le metriche sono implementati da zero in Python puro. Questo e' una scelta deliberata: il pieno controllo del codice permette di integrare qualsiasi tipo di logica di controllo nelle fasi successive.

2. Architettura del sistema

Il codice e' organizzato in due package Python principali: **simulator/** (il simulatore di rete vero e proprio, usato in produzione) e **eds/** (motore generico di riferimento, mantenuto per scopo didattico). Il package **simulator/** e' quello con cui si lavora.

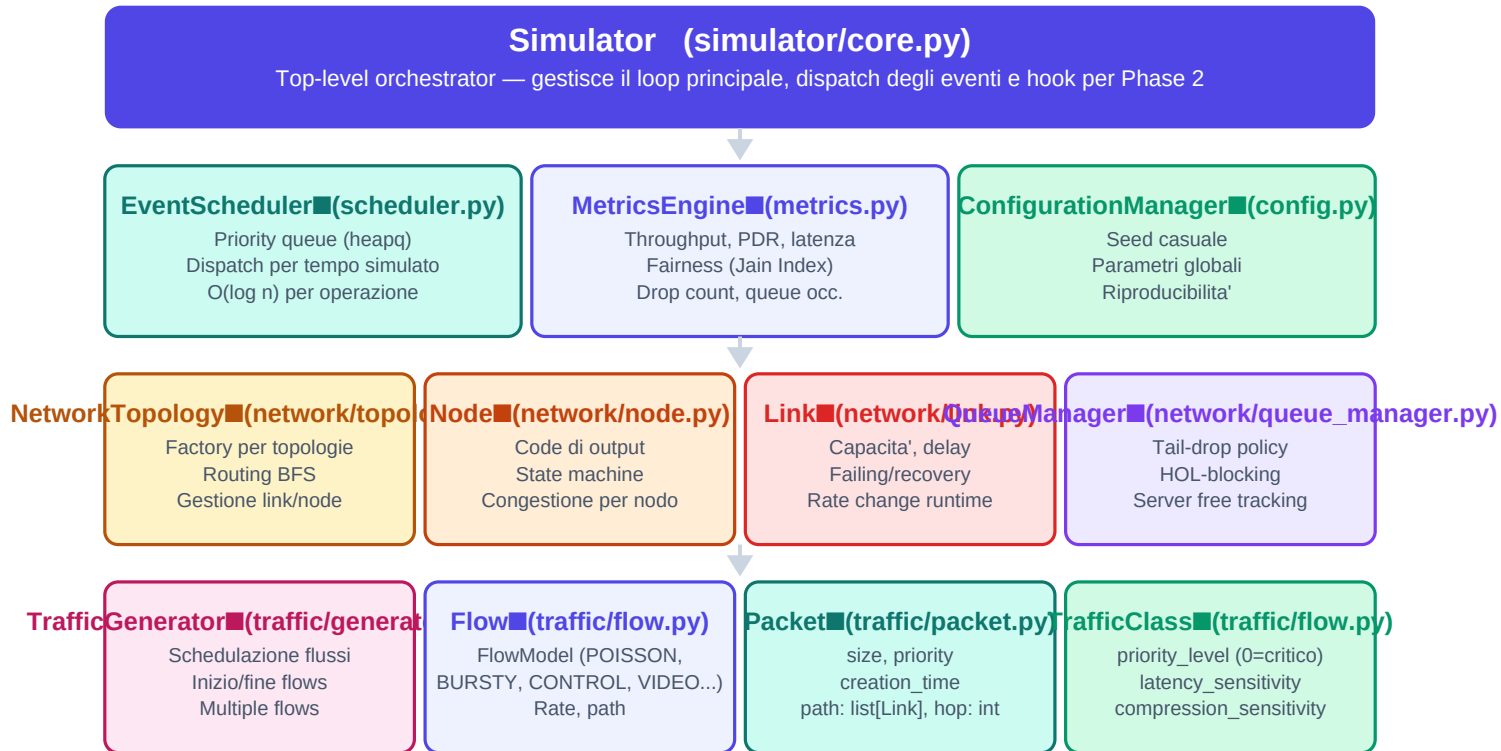


Fig. 2 — Architettura a livelli del simulatore. Le frecce indicano le dipendenze principali.

Modello di servizio: M/D/1

Ogni nodo della rete implementa un modello di coda **M/D/1**:

M — arrivi Markoviani	I pacchetti arrivano secondo un processo di Poisson con tasso λ (pkt/s). Il tempo inter-arrivo e' distribuito esponenzialmente.
D — servizio deterministico	Il tempo di servizio e' costante: $1/\text{service_rate}$ secondi per pacchetto. Il server sa esattamente quando sara' libero.
/1 — un server	Ogni output port ha un unico server. I pacchetti si accodano in FIFO. Il campo <code>server_free_at</code> nel <code>QueueManager</code> traccia quando il server si libera.

Nota: HOL-blocking: se il server e' occupato, il pacchetto in testa alla coda (Head-Of-Line) blocca tutti quelli successivi, anche se destinati a link diversi. Questo e' il comportamento realistico di un router.

3. Il motore a eventi discreti

Il **cuore** del simulatore e' l'EventScheduler: una priority queue (heap) ordinata per tempo simulato. Ad ogni passo, si estrae l'evento con timestamp piu' basso e lo si processa. Gli handler degli eventi possono schedulare nuovi eventi futuri. La simulazione termina quando la queue e' vuota o si raggiunge end_time.

I 14 tipi di evento (EventType)

Categoria	Tipo Evento	Descrizione
Traffico	FLOW_START	Attiva un flusso: schedula il primo PACKET_GENERATE
Traffico	FLOW_STOP	Ferma la generazione di nuovi pacchetti dal flusso
Traffico	PACKET_GENERATE	Crea un pacchetto e schedula il prossimo (interarrival time) + PACKET_ARRIVAL al primo hop
Traffico	PACKET_ARRIVAL	Pacchetto giunge a un nodo: se hop < len(path) -> QUEUE_ENQUEUE; altrimenti -> PACKET_DELIVER
Coda	QUEUE_ENQUEUE	Pacchetto entra nella coda di output; se coda piena -> PACKET_DROP; se server libero -> QUEUE_DEQUEUE immediato
Coda	QUEUE_DEQUEUE	Pacchetto esce dalla coda e va al link: schedula PACKET_TRANSMIT
Coda	PACKET_DROP	Pacchetto scartato (tail-drop o DROP_LOW_PRIORITY). Registrato in MetricsEngine.
Trasmissione	PACKET_TRANSMIT	Inizio trasmissione: dopo link.delay seconds schedula PACKET_ARRIVAL al nodo successivo
Trasmissione	PACKET_DELIVER	Pacchetto consegnato a destinazione. Registra latenza = t_deliver - t_create.
Rete	LINK_RATE_CHANGE	Cambia la capacita' del link a runtime (simula degradazione di banda)
Rete	LINK_FAILURE	Mette il link offline (active=False). I pacchetti che arrivano vengono droppati.
Rete	LINK_RECOVERY	Ripristina il link (active=True). I flussi riprendono normalmente.
Monitor	METRIC_SAMPLE	Campionamento periodico: crea un MetricSnapshot e lo aggiunge alla lista.
Monitor	STATE_UPDATE	Hook per Phase 2: il control plane puo' leggere lo stato e modificare parametri.

Ciclo di vita di un pacchetto

Dalla generazione alla consegna, ogni pacchetto attraversa una sequenza precisa di eventi. Il pacchetto porta con se' il campo `path: list[Link]` (calcolato via BFS al momento della creazione) e `hop: int` che avanza ad ogni `PACKET_ARRIVAL`.



Fig. 3 — Ciclo di vita di un pacchetto. Il drop avviene solo in fase di ENQUEUE (tail-drop) o nello stato DROP_LOW_PRIORITY.

Routing: BFS su grafo non pesato

Il percorso di ogni pacchetto viene calcolato **una sola volta** alla creazione del flusso tramite una visita BFS (Breadth-First Search) sul grafo della rete. Il path e' una lista ordinata di link da attraversare. Non esiste rerouting dinamico in Phase 1: se un link fallisce, i pacchetti su quel path vengono droppati finche' il link non si recupera. Phase 2 potra' implementare rerouting come azione correttiva.

4. Rete e gestione delle code

NetworkTopology e topologie disponibili

La classe NetworkTopology gestisce il grafo della rete (nodi e link) e fornisce metodi factory per creare topologie standard. In Phase 1 sono implementate:

<code>single_bottleneck(n_sources, capacity, queue_size)</code>	n sorgenti -> router -> destinazione. Il link router->dst e' il bottleneck. E' la topologia usata in tutti e 6 gli scenari canonici.
<code>mesh(n_rows, n_cols, capacity)</code>	Griglia rettangolare di nodi, ognuno connesso ai vicini. Utile per scenari con routing alternativo e failure parziali.

Node e QueueManager

Ogni nodo della rete ha una o piu' **code di output** (una per link uscente). Il QueueManager gestisce la coda FIFO con questi parametri:

<code>max_size</code>	Capacita' massima della coda in pacchetti. Se piena -> PACKET_DROP (tail-drop policy).
<code>service_rate</code>	Pacchetti serviti al secondo ($1/\text{service_rate}$ = tempo di servizio per pacchetto).
<code>server_free_at</code>	Timestamp in cui il server sara' libero. Permette di calcolare correttamente il tempo di dequeue anche con arrivi ravvicinati.
<code>queue_occupancy</code>	Proprieta' calcolata: $\text{len}(\text{queue}) / \text{max_size}$. Usata dalla CongestionStateMachine per decidere le transizioni di stato.

Link

La classe Link modella un collegamento fisico tra due nodi con questi attributi:

<code>capacity (pkt/s)</code>	Numero massimo di pacchetti trasmissibili al secondo. Determina il <code>service_rate</code> della coda di output.
<code>delay (s)</code>	Ritardo di propagazione del link (simulato come tempo tra PACKET_TRANSMIT e PACKET_ARRIVAL al nodo successivo).
<code>active (bool)</code>	Se False, il link e' down: tutti i pacchetti vengono droppati (simula guasto fisico).

Tail-drop e DROP_LOW_PRIORITY

Esistono due meccanismi di scarto dei pacchetti, distinti nel comportamento:

Tail-drop (coda piena)	Avviene in QUEUE_ENQUEUE quando <code>len(queue) >= max_size</code> . Il pacchetto viene scartato prima di entrare in coda, indipendentemente dalla sua priorit�. E' il meccanismo base di congestione.
DROP_LOW_PRIORITY (stato CongestionStateMachine)	Quando la coda supera il 95% di occupazione, i pacchetti con <code>priority > 0</code> (VIDEO, TELEMETRY) vengono scartati PRIMA di tentare l'enqueue. I pacchetti CONTROL (<code>priority=0</code>) passano sempre. Questo protegge il traffico critico durante sovraccarico estremo.

Nota: Una volta che un pacchetto e' in coda, e' al sicuro: non viene mai scartato dopo l'enqueue. Il drop e' solo all'ingresso.

5. Traffico e modelli di flusso

TrafficClass — classificazione del traffico

Ogni flusso appartiene a una TrafficClass che definisce le caratteristiche del tipo di traffico. Tre classi sono usate negli scenari canonici:

Classe	priority_level	Dimensione (byte)	latency_sensitive	compression_sensitive	Protezione DROP
CONTROL	0 (massima)	100 fisso	Si'	No	Non droppato in DROP_LOW_PRI
TELEMETRY	1	200-300	No	Si'	Droppato in DROP_LOW_PRI
VIDEO	2 (minima)	1400-1500	Si'	Si'	Droppato in DROP_LOW_PRI

Nota: La priorita' 0 e' la piu' ALTA (traffico critico). La priorita' 2 e' la piu' BASSA. In DROP_LOW_PRIORITY vengono droppati i pacchetti con priority > 0.

FlowModel — modelli di generazione del traffico

Il FlowModel determina come vengono schedulati i PACKET_GENERATE nel tempo:

Modello	Distribuzione inter-arrivo	Uso tipico
POISSON	Esponenziale (media 1/rate) — arrivi casuali indipendenti	Traffico generico, scenario 1
BURSTY	Alternanza burst/silenzio: burst=Poisson ad alto rate, poi pausa	Flash crowd, traffico video reale
CONTROL	Periodico fisso: ogni 1/rate secondi esattamente	Messaggi di controllo a frequenza fissa
VIDEO	Periodico con jitter gaussiano (5ms std)	Stream video a frame rate costante
PERIODIC_TELEMETRY	Periodico + probabilita' di pacchetto extra	Telemetria di sensori periodici

Il Packet

Ogni Packet e' un oggetto immutabile creato al momento del PACKET_GENERATE con questi campi fondamentali:

size (byte)	Dimensione del pacchetto. Usata per calcolare il tempo di trasmissione.
-------------	---

priority	Livello di priorit� (0=massima). Determina se il pacchetto sopravvive allo stato DROP_LOW_PRIORITY.
creation_time	Timestamp simulato al momento della generazione. Usato per calcolare la latenza end-to-end.
path: list[Link]	Lista dei link da attraversare dal source al destination. Calcolata via BFS e mai modificata.
hop: int	Indice del link corrente nel path. Incrementato ad ogni PACKET_ARRIVAL. Quando hop == len(path), il pacchetto � arrivato.
flow_id	ID del flusso di appartenenza. Usato dalla MetricsEngine per calcolare il Jain Fairness Index per flusso.

6. State machine di congestione

La `CongestionStateMachine` e' il meccanismo con cui ogni nodo reagisce alla congestione. Monitora l'occupazione della coda (`queue_occupancy`) e transita automaticamente tra 5 stati predefiniti. E' il punto di integrazione principale con Phase 2: il control plane puo' modificare le soglie o aggiungere azioni correttive tramite l'hook `_on_state_update`.

Soglie della `CongestionStateMachine`

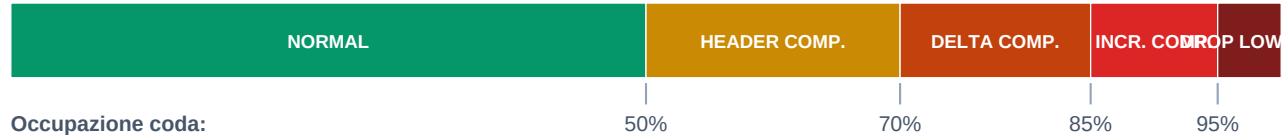


Fig. 4 — Soglie della `CongestionStateMachine`. Al superamento di ogni soglia, il nodo entra nello stato corrispondente.

Stato	Soglia occupazione	Azione corrente (Phase 1)	Scopo
NORMAL	< 50%	Pass-through — nessuna azione	Rete non congestionata, operazione normale
HEADER_COMPRESSION	>= 50%	Placeholder: <code>compression_ratio=1.0</code> (Phase 2)	Segnale precoce: inizia a ridurre overhead header
DELTA_COMPRESSION	>= 70%	Placeholder: <code>compression_ratio=1.0</code> (Phase 2)	Invia solo le differenze rispetto al pacchetto precedente
INCREMENTAL_COMPRESSION	>= 85%	Placeholder: <code>compression_ratio=1.0</code> (Phase 2)	Compressione aggressiva — riduzione massima dati
DROP_LOW_PRIORITY	>= 95%	DROP pacchetti con <code>priority > 0</code> prima dell'enqueue	Protegge il traffico critico (CONTROL) da blackout totale

Logica di transizione

Il metodo `evaluate(occupancy)` determina lo stato target in modo deterministico: scansiona le soglie in ordine crescente e restituisce l'ultimo stato superato. Il metodo `update(occupancy, sim_time)` chiama `evaluate` e poi `transition()` se lo stato e' cambiato, registrando la transizione nella `history`.

`CongestionStateMachine.update(occupancy=0.88, sim_time=42.5)`
Nell'esempio sopra: `0.88 >= 0.85 (INCREMENTAL_COMPRESSION)` ma `< 0.95 (DROP_LOW_PRIORITY)`. Lo stato diventa `INCREMENTAL_COMPRESSION`. Se prima era `DELTA_COMPRESSION`, viene registrata una transizione e incrementato `_state_transitions` (usato come metrica).

Soglie configurabili

Le soglie di default possono essere sovrascritte passando un dizionario `thresholds` al costruttore. Phase 2 potrà modificarle a runtime tramite l'hook `STATE_UPDATE` per adattare il comportamento alle condizioni di rete osservate.

DEFAULT_THRESHOLDS	HEADER_COMPRESSION: 0.50 DELTA_COMPRESSION: 0.70 INCREMENTAL_COMPRESSION: 0.85 DROP_LOW_PRIORITY: 0.95
Configurazione custom	<code>CongestionStateMachine(thresholds={CongestionState.DROP_LOW_PRIORITY: 0.80})</code> — abbassa la soglia di drop per reagire prima

7. Metriche di performance

La MetricsEngine raccoglie dati durante la simulazione e produce campioni periodici (MetricSnapshot). Ogni snapshot e' uno snapshot completo dello stato della rete al momento del campionamento. L'intervallo e' configurabile (default: ogni 10 secondi simulati).

1. Throughput

$$\text{Throughput} = \text{pacchetti_consegnati_nell'intervallo} / \text{delta_t} \quad [\text{pkt/s}]$$

Misura quanti pacchetti vengono effettivamente consegnati per secondo nell'intervallo tra due campionamenti. **Diverso dal rate di input:** in condizioni di congestione, il throughput e' limitato dalla capacita' del bottleneck, non dal rate di generazione. **Vuoi:** alto (vicino alla capacita' del link).

2. Packet Delivery Ratio (PDR)

$$\text{PDR} = \text{pacchetti_consegnati} / \text{pacchetti_generati} \text{ in } [0, 1]$$

La metrica piu' importante per valutare la qualita' del servizio. Un PDR=1.0 significa zero drop. Un PDR=0.65 significa che il 35% dei pacchetti e' andato perso. Tiene conto di TUTTI i drop: tail-drop, DROP_LOW_PRIORITY e link failure. **Vuoi:** piu' vicino a 1.0 possibile.

3. End-to-End Latency (latenza media)

$$\text{Latency} = \text{SOMMA}(t_deliver - t_create) / \text{pacchetti_consegnati} \quad [\text{secondi}]$$

Latenza media dalla generazione del pacchetto alla sua consegna. Include il tempo in coda (dominante sotto congestione), il tempo di trasmissione sul link e il delay di propagazione. Sotto forte congestione con coda piena (18-19 pacchetti x 0.1s/pkt = 1.8s), la latenza puo' diventare molto alta. **Vuoi:** bassa. Critica per traffico latency-sensitive (CONTROL, VIDEO).

4. Queue Occupancy (occupazione della coda)

$$\text{Queue_Occ} = \text{media}(\text{len}(q_i) / \text{max_q_i}) \text{ su tutte le code in } [0, 1]$$

Media dell'occupazione relativa di tutte le code di tutti i nodi. E' il dato di input della CongestionStateMachine: quando supera le soglie (50%, 70%, 85%, 95%), il nodo cambia stato. **Attenzione:** la media include anche le code dei nodi sorgente e destinazione (quasi sempre vuote), quindi l'occupazione reale del router puo' essere piu' alta della media globale. **Vuoi:** bassa (idealmente < 50% per rimanere in stato NORMAL).

5. Fairness Index di Jain

$$J = (\text{SOMMA}_i(x_i))^2 / (n * \text{SOMMA}_i(x_i^2)) \text{ in } (0, 1]$$

Misura quanto equamente la banda e' distribuita tra i flussi attivi. x_i e' il numero di pacchetti consegnati del flusso i , n e' il numero di flussi. $J=1.0$ significa distribuzione perfettamente equa; $J=0.5$ significa forte

squilibrio. **Vuoi:** 1.0 (equita' massima tra flussi).

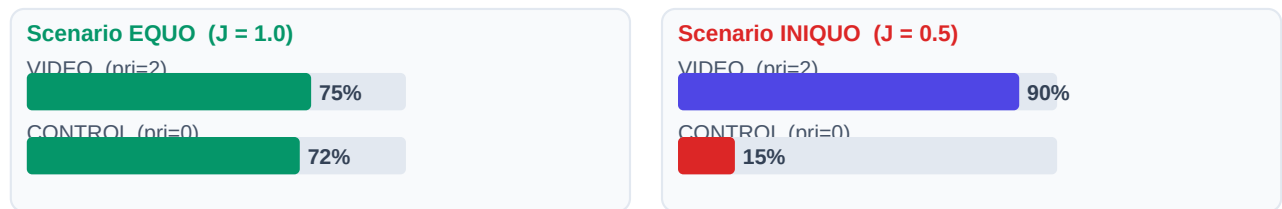


Fig. 5 — Confronto fairness: a sinistra 2 flussi con consegne bilanciate ($J=1.0$); a destra un flusso dominante ($J=0.5$).

6. Drop Count e State Transitions

Drop Count	Contatore cumulativo di tutti i pacchetti droppati. Non distingue tra tail-drop e DROP_LOW_PRIORITY in Phase 1 (Phase 2 potrà separare le cause).
State Transitions	Numero di volte che la CongestionStateMachine ha cambiato stato. Molte transizioni indicano oscillazione: la rete e' instabile al confine tra due stati.

La struttura MetricSnapshot

Ogni campionamento produce un oggetto MetricSnapshot con tutti i valori al momento del campionamento:

```
MetricSnapshot( time=42.5, # tempo simulato throughput=8.3, # pkt/s nell'ultimo
intervallo packet_delivery_ratio=0.64, # 64% consegnati end_to_end_latency=1.72, # 1.72
secondi medi queue_occupancy=0.91, # 91% media code drop_count=127, # pacchetti droppati
totali fairness=0.87, # Jain index congestion_state_transitions=14 # cambi di stato
totali )
```

8. Scenari canonici (6 test)

Tutti e 6 gli scenari usano la topologia **single_bottleneck**: N sorgenti convergono su un router centrale, collegato a una destinazione tramite un link con capacita' limitata. E' la topologia piu' semplice che produce congestione in modo controllato e analiticamente comprensibile.

#	Nome	Load / Cap	Eventi iniettati	Durata	Focus
1	Single Bottleneck	13 / 10 pkt/s (rho=1.3)	Nessuno	60s	Drop sistemici, PDR stabile
2	Flash Crowd	6 / 10 -> 12/10 a t=20	FLOW_START t=20, FLOW_STOP t=50	80s	Risposta a surge improvviso
3	Bandwidth Degradation	12 / 10 -> 4 pkt/s a t=30	LINK_RATE_CHANGE t=30 (cap=4), t=60 (cap=10)	90s	Degradazione banda, recovery
4	Link Failure & Recovery	9 / 10 pkt/s	LINK_FAILURE t=30, LINK_RECOVERY t=55	90s	Impatto blackout, velocita' recovery
5	Persistent Overload	15 / 10 pkt/s	Nessuno	100s	Comportamento a regime, stato stazionario
6	Mixed Telemetry+Video	11 / 10 pkt/s	Nessuno	80s	Protezione CONTROL vs VIDEO/TELEMETRY

Scenario 1 — Single Bottleneck (analisi dettagliata)

E' lo scenario di riferimento, usato per validare il simulatore contro la teoria delle code M/D/1. Configurazione:

Sorgente src0	Flow VIDEO, FlowModel.POISSON, rate=8.0 pkt/s, priority=2
Sorgente src1	Flow CONTROL, FlowModel.CONTROL, rate=5.0 pkt/s, priority=0
Load totale	8 + 5 = 13 pkt/s > bottleneck 10 pkt/s -> rho=1.3 (overloaded)
Coda	max_size=20 pacchetti, service_rate=10 pkt/s
Link delay	10ms (propagation delay, trascurabile rispetto al queueing delay)

Scenario 6 — Mixed Traffic (protezione prioritaria)

Questo scenario verifica il comportamento del DROP_LOW_PRIORITY: 3 flussi con priorit  diverse su una rete moderatamente overloaded ($11 > 10$ pkt/s). Il traffico CONTROL (priority=0, rate=2 pkt/s) deve sopravvivere anche quando la coda supera il 95%. VIDEO e TELEMETRY vengono droppati per primi.

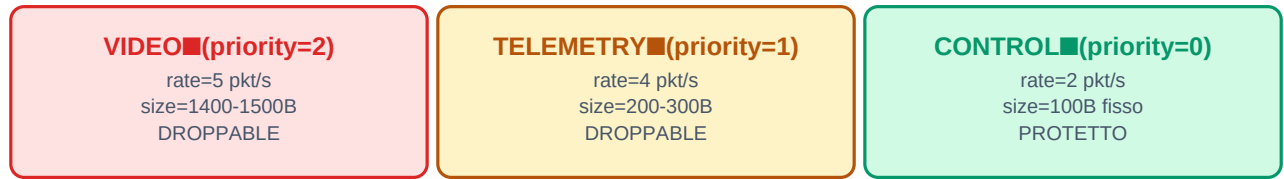


Fig. 6 — Scenario 6: 3 classi di traffico con priorit  diverse su bottleneck 10 pkt/s.

9. Risultati e validazione Phase 1

PDR comparativo tra scenari

I valori indicativi di PDR per ogni scenario (seed fisso, run deterministico):

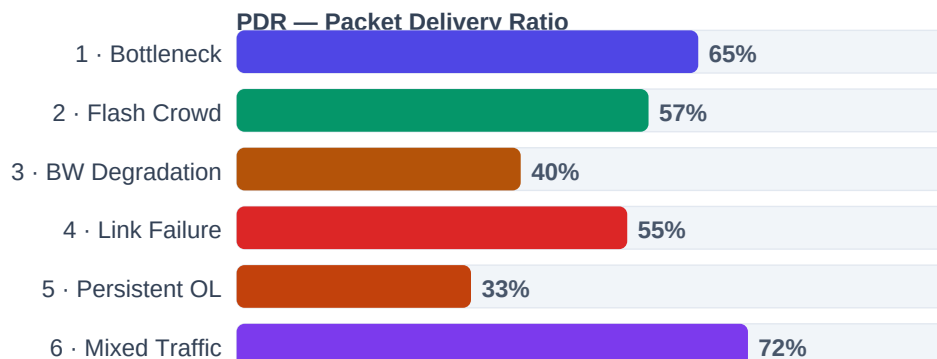


Fig. 7 — PDR per scenario. Lo scenario 5 (persistent overload con $\rho=1.5$) ha il PDR piu' basso; lo scenario 6 beneficia della protezione del CONTROL.

Validazione M/D/1 — Scenario 1

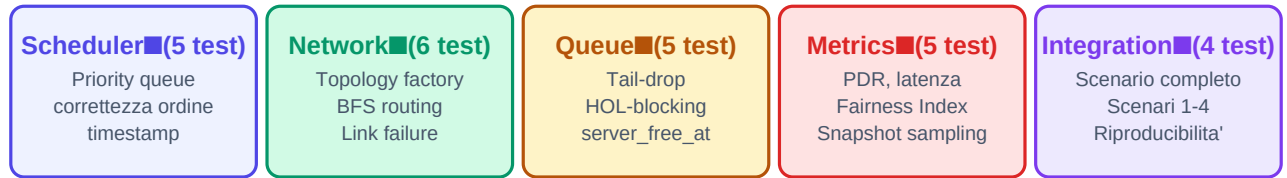
Il simulatore e' stato validato confrontando i risultati con la teoria analitica per code M/D/1 in condizione di overload ($\rho > 1$).

lambda (tasso arrivi)	13 pkt/s (8 VIDEO + 5 CONTROL)
mu (tasso servizio)	10 pkt/s (capacita' bottleneck)
$\rho = \text{lambda}/\mu$	1.3 -> sistema instabile, coda cresce indefinitamente
Coda max	20 pacchetti -> si riempie rapidamente
Latenza attesa (teoria)	$\sim 18-19 \text{ pkt in coda} \times 0.1\text{s}/\text{pkt} = 1.8 \text{ secondi}$
Latenza simulata	$\sim 1.7 \text{ secondi}$ (coerente con teoria)
Stati osservati	NORMAL -> DELTA_COMP. -> INCR_COMP. -> DROP_LOW_PRI (oscillazione)
PDR osservato	~ 0.65 (35% dei pacchetti droppato, principalmente VIDEO priority=2)

Nota: L'oscillazione tra INCREMENTAL_COMPRESSION e DROP_LOW_PRIORITY a regime e' attesa: quando la coda supera il 95% i low-priority vengono droppati, la coda si svuota leggermente, torna sotto la soglia, e i drop si fermano finche' la coda non si riempie di nuovo.

Test suite — 25 test, tutti passati

Il simulatore e' validato da una suite di 25 test unitari che coprono tutti i componenti principali:



10. Prossimi passi — Phase 2 e oltre

Hook di integrazione: `_on_state_update`

Il simulatore e' progettato per essere esteso. Il metodo `_on_state_update` in `core.py` viene chiamato ad ogni evento `STATE_UPDATE`. In Phase 1 e' vuoto. Phase 2 inserira' qui la logica di controllo:

```
def _on_state_update(self, event: Event) -> None: # Phase 1: nessuna azione # Phase 2:
    leggere metriche, modificare soglie, cambiare rate state =
    self.topology.get_node("router").congestion_state if state ==
    CongestionState.DROP_LOW_PRIORITY: # Rule: abbassa il rate del flusso piu' aggressivo ...
```

Phase 2 — Rule-Based Control

Il control plane basato su regole leggerà lo stato della rete (`CongestionState`, `queue_occupancy`, `throughput`) e applicherà azioni correttive predefinite: throttling dei flussi low-priority, modifica dinamica delle soglie, rerouting su link alternativi.

Phase 3 — Multi-Agent Reinforcement Learning (MARL)

Ogni router diventa un agente RL che impara la policy ottimale di gestione della congestione tramite trial-and-error nel simulatore. Il reward function e' definito in termini delle metriche già implementate: PDR, latenza, fairness.

Phase 4 — Agentic AI

Un LLM (Claude o simile) con accesso agli strumenti di osservazione e controllo della rete decide autonomamente le azioni da intraprendere, spiegando le sue decisioni in linguaggio naturale. Il simulatore event-driven rimane la piattaforma di esecuzione.

Punto di forza del design

In tutte e quattro le fasi, si utilizza lo stesso simulatore — lo stesso codice, gli stessi scenari, le stesse metriche. Questo garantisce confronti rigorosi tra approcci.